DAY 1 · MODULE 2

Source Control & Branching Strategy in Azure Repos

Choosing a branching model, reviewing changes through pull requests, and protecting main with branch policies.

Azure DevOps, CI/CD and AI-Assisted Release Readiness

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

ROADMAP

What We'll Cover

1. Git Fundamentals in Azure Repos
2. Branching Models
3. Pull Requests & Code Review
4. Branch Policies & Status Checks
5. Merge Strategies & History
6. Protecting Main & What's New (opt.)
7. Case Study
8. Knowledge Check
9. Recap

One thread runs through the module: bug #57 travels from a branch to a protected main – only through a reviewed, linked, validated pull request.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

1

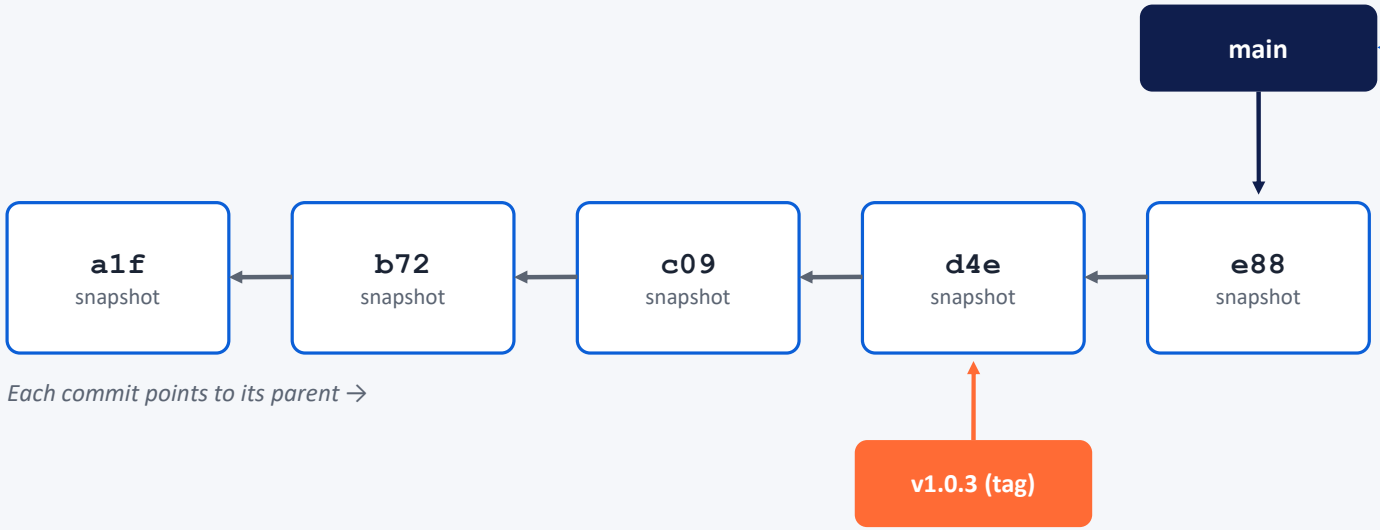
Git Fundamentals in Azure Repos

Just enough Git internals to make good branching and merging decisions

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

THE MODEL

A Commit Is a Snapshot; a Branch Is Just a Pointer



Consequences

Commits never change – amend, rebase and squash create new ones.

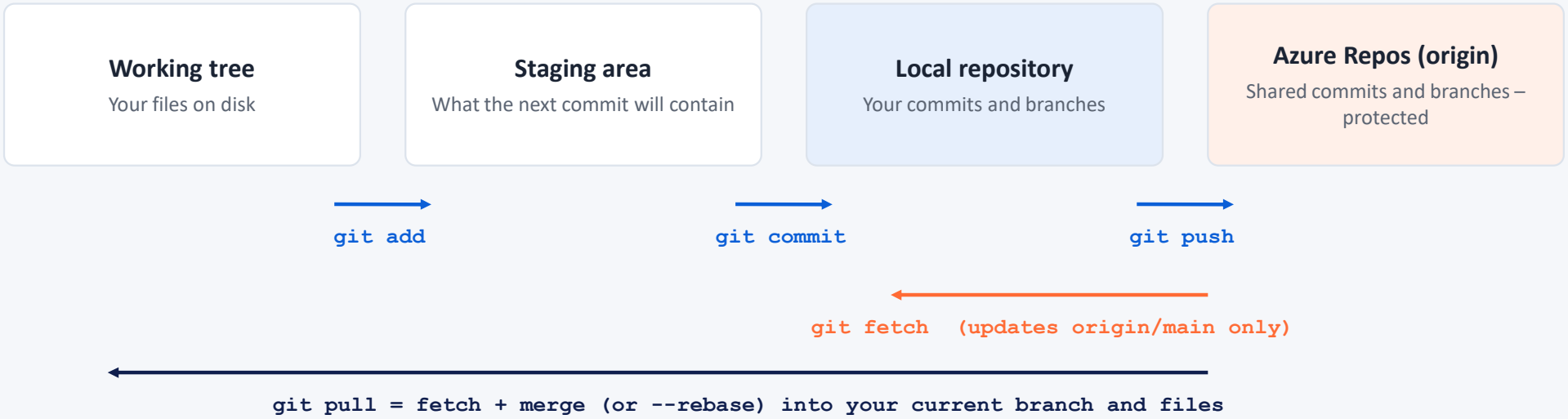
Branches are cheap names.

Tags mark releases.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

LOCAL AND REMOTE

Your Clone and Azure Repos Are Two Separate Copies



Only pushed commits exist for the team – and the server, not your laptop, enforces branch protection.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

WHAT THE PLATFORM ADDS

Azure Repos: Git Plus Identity, Policy and Traceability

1

Repositories
Unlimited private Git repos per project, each with its own settings

2

Identity & permissions
Entra ID users and groups; rights per repo and per branch

3

Pull requests
Reviewers, votes, threads, suggestions, auto-complete

4

Branch policies
Rules a PR must satisfy before it can complete

5

Links to Boards & Pipelines
#ID mentions, linked work items, build validation

6

Security add-ons
Secret Protection & Code Security for Azure DevOps

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

WORKING SET

The Git Commands This Module Relies On

Command	What it does	When we use it
git switch -c feature/57-…	Create and switch to a new branch	Start work from a fresh main
git fetch --prune	Update origin/* and drop deleted branches	Before comparing with the server
git pull --rebase	Fetch, then replay your commits on top	Keep your branch current without merge noise
git push -u origin <branch>	Publish the branch and set upstream	Before opening a pull request
git push --force-with-lease	Overwrite your own remote branch safely	Only after tidying your own branch
git rebase -i origin/main	Reorder, squash or reword your commits	Tidy commits before review
git revert <sha>	New commit that undoes an old one	Undo something already on main
git cherry-pick -x <sha>	Copy a commit to another branch, noting its origin	Hotfix into a release branch
git log --oneline --graph --all	Draw the commit graph	Understand what a merge did

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

SCALE

Large Repositories Need Different Habits

Partial clone	Sparse checkout	Shallow clone	Git LFS
`git clone --filter=blob:none` History now, file contents on demand	`git sparse-checkout set src/payments` Only the folders you work on	`fetchDepth: 1 (pipelines)` For build agents – set 0 when tools need history	`git lfs track "*.bin"` Large binaries outside normal Git objects

New in 2026: Azure Repos removed the 100-million-object limit per repository – very large monorepos no longer hit a hard ceiling, but the habits above still decide how fast they feel.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

REPOSITORY SETTINGS

Five Repository Settings Worth Checking on Day One

1	**Commit mention linking**	#ID in commits creates links – On
2	**Mention-based resolution**	‘Fixes #57’ changes state – usually Off
3	**Default branch**	main – used by policies and Compare
4	**Branch security on main**	Deny force push; bypass for nobody day to day
5	**Branch creation & forks**	Who may create branches, and where

Permissions decide who can act; policies decide what a pull request must satisfy. You need both.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

2 Branching Models

Trunk-based development vs GitFlow – and how to choose for your team

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

WHY IT MATTERS

The Longer a Branch Lives, the Harder It Is to Merge

Hours — Tiny conflicts · quick, focused review · fast feedback

Days — Manageable conflicts · reviews start to slip

Weeks — **Merge projects · rushed reviews · late surprises in test**

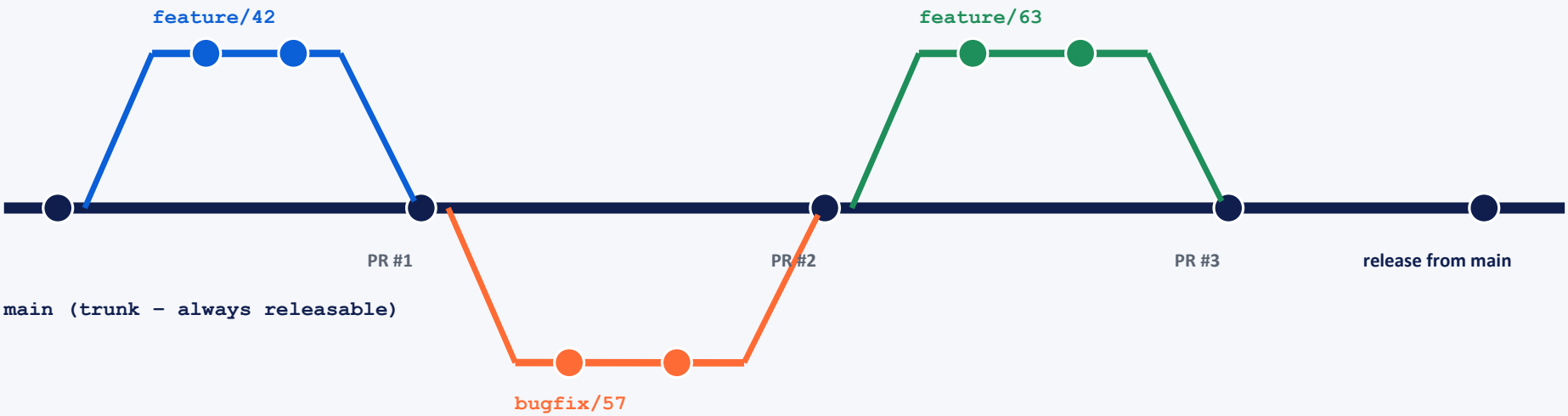
branch age → merge cost

DORA's trunk-based capability: ≤ 3 active branches, merged to trunk at least daily, no code freezes.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

MODEL 1

Trunk-Based Development: Short Branches, One Trunk



Branch life: Hours to ~2 days

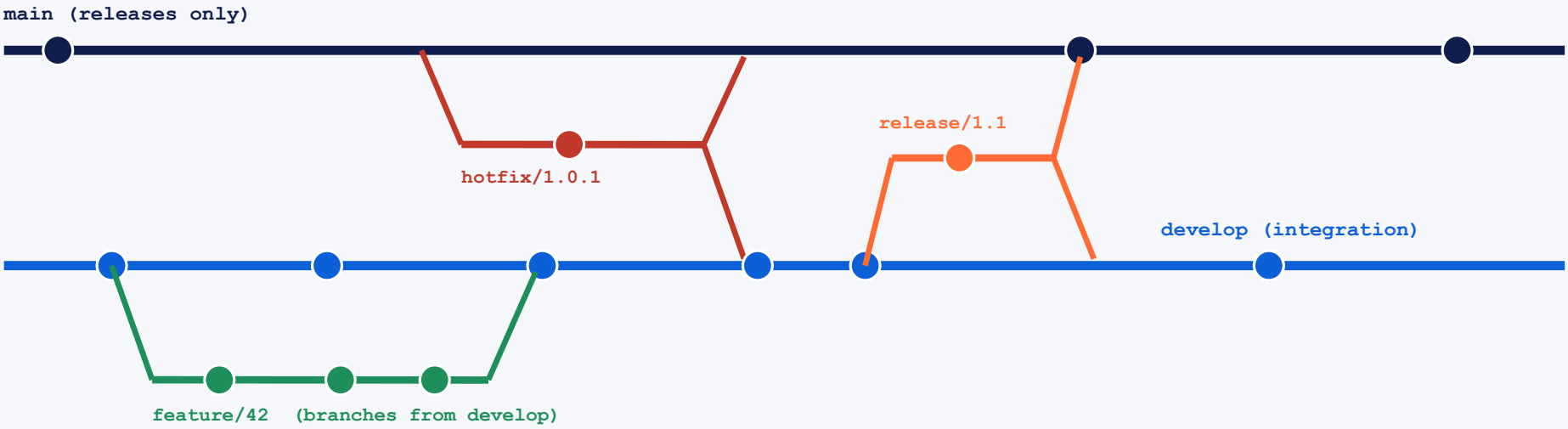
Merges: Small, through PRs

Needs: Fast CI on every PR, feature flags

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

MODEL 2

GitFlow: Parallel Long-Lived Branches for Versioned Releases

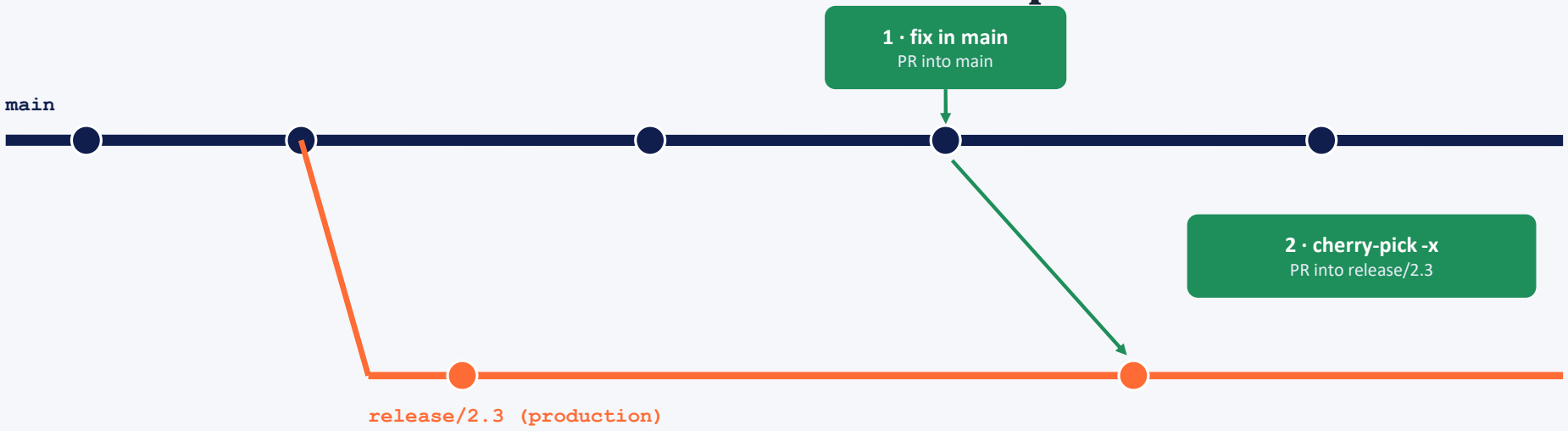


Fits versioned products and parallel supported versions. Costs: every change crosses two or more branches, and work waits in develop.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

THE MIDDLE PATH

Release Branches in a Trunk-Based Team: Fix Upstream First



Upstream first: fix main, then cherry-pick. Fixing only the release branch lets the next release from main bring the bug back.

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

SIDE BY SIDE

Trunk-Based, GitHub Flow and GitFlow Compared

	Trunk-based	GitHub flow	GitFlow
Long-lived branches	main (+ short release/x)	main	main + develop (+ release, hotfix)
Feature branch life	Hours to ~2 days	Days	Days to weeks
Release from	main, or release/x	main	release branch → main, tagged
Hotfix path	Fix main, cherry-pick to release	PR into main, redeploy	hotfix from main → main + develop
Best fit	Continuous delivery, strong CI	Small teams, web apps	Versioned products, many versions
Main risk	Unfinished work without flags	Same as trunk-based	Merge debt, forgotten back-merges

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CHOOSING

Five Questions That Pick a Branching Model

1	**Release cadence?**	Daily/weekly → trunk-based · scheduled versions → release branches
2	**Versions supported at once?**	One → trunk-based · several → release branches (maybe GitFlow)
3	**Is a green PR build trustworthy?**	If not, fix the tests first – not the branching model
4	**Can unfinished work merge safely?**	Feature flags / dark launches make short branches easy
5	**Who approves releases?**	Prefer per-environment approvals with evidence (Module 5)

Default: trunk-based with short branches into a protected main – move away only when a question forces you to.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

DECOUPLE DEPLOY FROM RELEASE

Feature Flags Let Unfinished Work Merge Safely

```
# illustrative: coupons merged early, switched off
from featureflags import is_enabled

def checkout_total(order, coupon=None):
    total = order_total(order.amount)
    if coupon and is_enabled("coupons", order.customer_id):
        total = apply_coupon(total, coupon)  # new
    return total        # old path stays the default
```

Merge — Small PRs into main, flag off

Deploy — Code in production, not visible

Release — Flag on: staff → 10% → all

Remove — Delete flag and old path

Every flag needs an owner and a removal date.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CONVENTIONS

Branch Names That Say What the Branch Is For

`feature/42-inactive-customer`	New work, linked to story 42
`bugfix/57-discount-threshold`	Defect fix, linked to bug 57
`release/2.3`	Cut from main at release; same policies via release/*
`hotfix/57-…`	Only if your model uses hotfix branches
`users/durai/spike-cache`	Personal experiments, outside shared folders

Folders (feature/, bugfix/, release/) can carry their own branch permissions and wildcard policies.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

3

Pull Requests & Code Review

The single, reviewed, traceable way into main

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

ANATOMY

Everything a Pull Request Carries in Azure Repos

1 **Title & description**
What, why, how tested – the reviewer's briefing (PR template)

2 **Work items**
Link to the story or bug – required by policy

3 **Reviewers**
Required and optional people or groups; each votes

4 **Files & threads**
Line comments, suggestions applied in one click

5 **Updates**
Each push is an iteration – compare any two

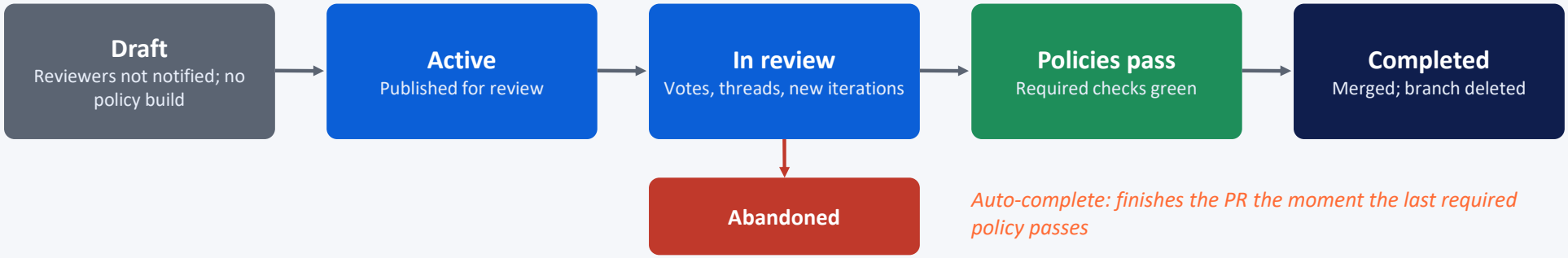
6 **Checks & policies**
Build, status checks, reviewers, comments – decides Complete

Complete: choose the merge type, delete the source branch, complete linked work items – or set auto-complete to finish when every policy passes.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

LIFECYCLE

From Draft to Completed: The Pull Request Lifecycle



Auto-complete: finishes the PR the moment the last required policy passes

Reviewer votes

Approve	Approve with suggestions	No vote	Wait for author	Reject

With the minimum-reviewers policy, Wait for author and Reject block completion (unless the policy allows it).

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

SIZE

Small Pull Requests Get Better Reviews

~50 lines  Real questions, real defects found

~400 lines  Upper limit for a careful single-sitting review

1,500+ lines 

- One logical change per PR
- Refactoring and behaviour changes in separate PRs
- Stack PRs: model → API → UI, merged behind a flag
- Call out generated and lock files in the description

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

REVIEWING

What a Reviewer Looks For – In This Order

1	**Intent**	Does it do what the work item asks – and only that?
2	**Correctness**	Edge cases, boundaries, error handling
3	**Tests**	Would a test fail if this code were wrong?
4	**Security & data**	Input validation, secrets, permissions, personal data in logs
5	**Design & readability**	Right place, clear names, understandable next year
6	**Style**	Only what the linter can't catch – automate the rest

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

COMMUNICATING

Comments That Get Changes Made, Not Arguments

Hard to act on

"This doesn't look right."

Easy to act on

"blocker: an order of exactly 1000 gets no discount – the story says '1000 or more'. Suggest '>=' (suggestion attached)."

Label every comment

blocker | issue | suggestion | question | nit | praise

Use suggestions for concrete code changes (applied in one click). Every thread ends Resolved, Won't fix or Closed – the comment-resolution policy blocks completion while any thread is Active.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

4

Branch Policies & Status Checks

Letting Azure Repos enforce what the team agreed

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

OVERVIEW

Six Policy Types – Five Configured in the Lab

1. **Minimum reviewers**
   Approvals, self-approval rules, what new pushes reset
2. **Linked work items**
   No work item, no merge – traceability enforced
3. **Comment resolution**
   Every thread resolved before completion
4. **Limit merge types**
   Which merge strategies are allowed
5. **Build validation**
   A pipeline must pass on the PR's merge result
6. **Status checks · demo**
   Needs a service that posts a status

Any required policy also blocks direct pushes: TF402455 "Pushes to this branch are not permitted; you must use a pull request."

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

POLICY 1

Minimum Reviewers: The Options That Actually Matter

Option	Lab setting	Why
Minimum number of reviewers	1 (2 for critical repos)	Four-eyes principle
Allow requestors to approve their own changes	Off	The author's approval mustn't count
Prohibit the most recent pusher from approving	On	Closes the 'push then approve' loophole
Allow completion if reviewers wait or reject	Off	'Wait for author' really means wait
When new changes are pushed	Reset all approval votes	Any change after approval is re-reviewed

Lighter alternative: "Require at least one approval on the last iteration".

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

POLICIES 2 AND 3

Linked Work Items and Resolved Comments

Check for linked work items · Required

No linked work item → Complete disabled.

On completion: the work item gets the PR, the merge commit and the build; the squash commit message gets “Related work items: #57”.

Check for comment resolution · Required

Every thread must be Resolved, Won’t fix, Closed or By design.

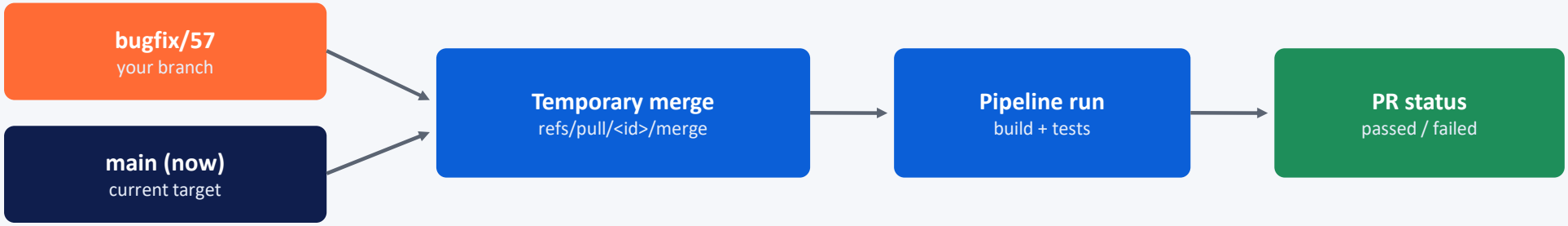
No ‘approved with open questions’ – and in Module 7, every AI review comment needs a human decision.

```
Merged PR 14: Apply discount at the threshold amount
Related work items: #57
```

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

POLICY 5

Build Validation Tests the Result of the Merge

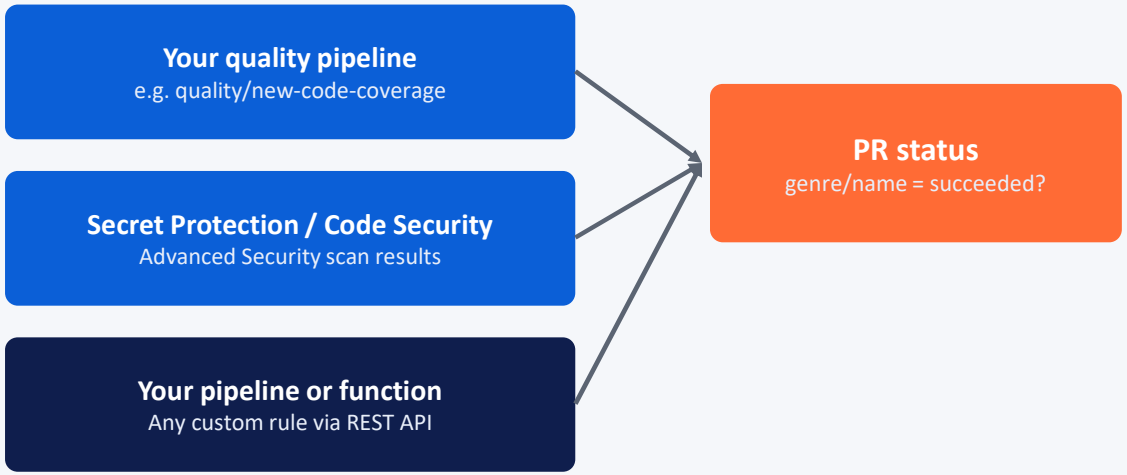


Setting	Options	Lab
Trigger	Automatic · Manual	Automatic
Policy requirement	Required · Optional	Required
Build expiration	Immediately · After n hours · Never	12 hours
Path filter	e.g. exclude /docs/*	None

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

POLICY 6

Status Checks Let Other Tools Gate a Pull Request



Option	Meaning
Requirement	Required or Optional
Authorized identity	Only accept the status from this identity
Reset conditions	Reset when the source branch updates
Applicability	Apply by default, or Conditional (once posted)
Path filter	Only for certain paths

Lab: demonstration only – a status check needs a service that posts statuses.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

SCOPE AND EXCEPTIONS

Where Policies Apply – and Who Can Bypass Them

One branch

Repos › Branches › main › Branch policies

A pattern

release/* – repository Policies tab

Every repo

Project settings › Repositories › Policies

Automatically included reviewers

Add a person or group when certain paths change.

Example: platform team, Required, path /infra/*

+ minimum reviewers = 1 for everything else.

Bypass permissions

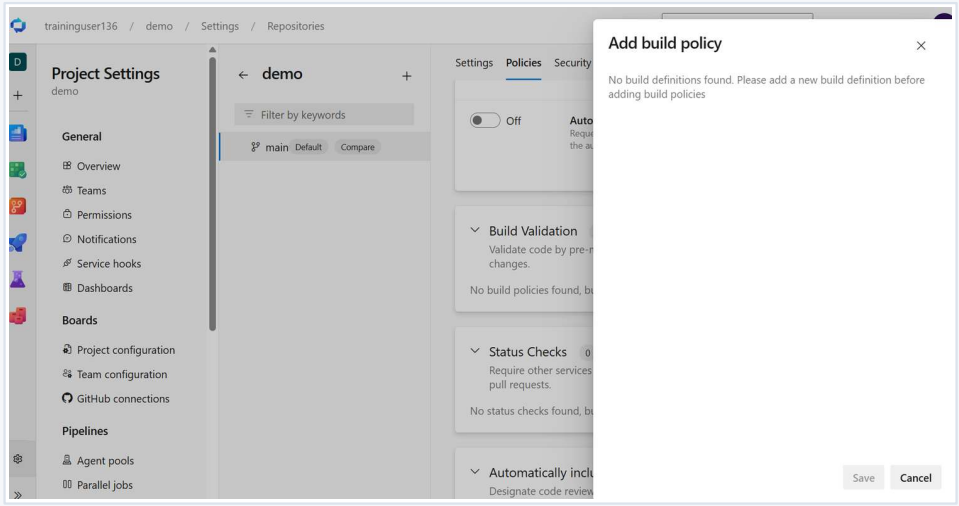
‘Bypass policies when completing pull requests’ and ‘… when pushing’.

Recorded with a reason – keep for a small break-glass group only.

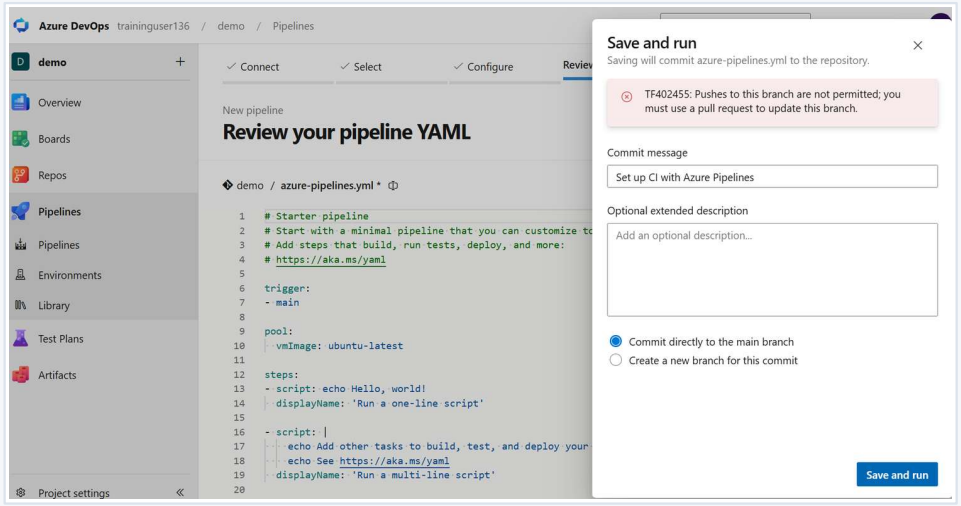
© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

FROM THE LAB VALIDATION

What the Policies Look Like in Your Organisation



Branch policies › Build validation › Add build policy



TF402455 – direct commit to a protected main is refused

Screens from the lab environment validation, 23 October 2026.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

5

Merge Strategies & History Hygiene

What completing a pull request does to main’s history

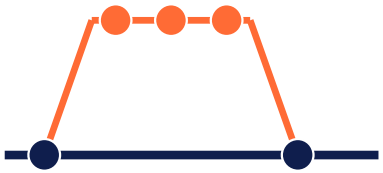
© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

FOUR OPTIONS

Same Pull Request, Four Different Histories

Basic merge (no FF)



Merge commit with two parents; all branch commits land on main

Squash merge



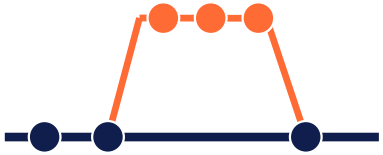
One new commit per PR; branch commits stay in the PR

Rebase + fast-forward



Commits replayed on main; main moves forward

Rebase with merge commit



Rebased commits + a merge commit marking the PR

Lab choice: squash – one reviewed, linked commit per pull request on main.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CHOOSING

Pick a Merge Type for What You Need From History

Need	Basic merge	Squash	Rebase + FF	Semi-linear
Linear, readable main	No	Yes	Yes	Mostly
Revert a whole PR in one step	revert -m 1	Yes – one commit	No – many commits	revert -m 1
Every branch commit kept on main	Yes	No – kept in the PR	Yes	Yes
Each commit on main passed PR checks	Merge commit only	Yes	Only the last	Merge commit only
One hop to PR and work item	Merge commit	Yes – message	No	Merge commit

Enforce the choice with the "Limit merge types" policy – decide once, not at every Complete.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

HYGIENE

Five Rules for a History You Can Trust

1	**Never rewrite shared history**	On main or shared branches, undo with git revert
2	**Tidy your own branch before review**	git rebase -i, then push --force-with-lease
3	**Messages for the reader in a year**	Imperative subject, the why, and #57
4	**Stay current with main daily**	Rebase or merge main into your branch
5	**Clean up and tag**	Delete merged branches; tag releases on main

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

CONFLICTS

Resolving Conflicts Without Breaking Main

```
# resolve on your branch, never on main
git fetch origin
git switch bugfix/57-discount-threshold
git rebase origin/main
#  ... CONFLICT in orders.py ...
#  edit orders.py, keep the right lines
git add orders.py && git rebase --continue
pytest -q                       # re-test
git push --force-with-lease     # own branch
```

In the browser

Small conflicts: the Merge Conflict extension's Conflicts tab – take source, take target or edit.

Either way

The resolved code is new and unreviewed – votes reset and build validation runs again.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

6

Protecting Main & What's New

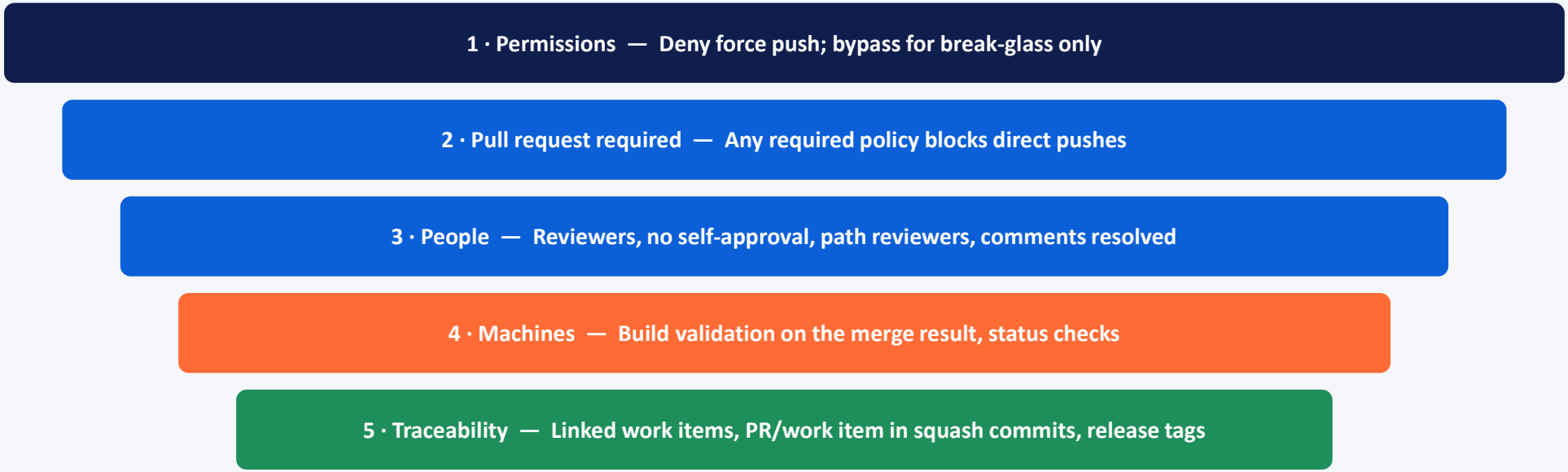
Defence in depth for main – plus optional awareness: AI arriving in the pull request

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

DEFENCE IN DEPTH

Five Layers Stand Between a Change and Main



Under every layer: an audit trail of PRs, votes, policy results and bypasses.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

OPTIONAL · WHAT'S NEW IN 2026

AI Reviews Arrive in Azure Repos Pull Requests

Jun 2026	Copilot code review for Azure Repos – technical / limited preview
26 Aug 2026	Copilot code review – public preview; branch-policy triggered, billed per review to Azure
30 Sep 2026	Effort levels (Lite / Balanced), resolve-all on applied suggestions, audit events
Jul 2026	Pull request status shown on Azure Boards cards (Sprint 277)
5 Aug 2026	Remote Azure DevOps MCP Server GA – AI agents read PRs under your identity
2026	Git 100-million-object repository limit removed

AI review comments are still comments: with comment resolution required, a human decides on every one.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

7

Case Study

Configure branch policies and drive a change through review to a protected main

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CASE STUDY

Protect Main and Fix the Discount Bug Through a Pull Request

1

Add your lab buddy (Basic, Contributors); create the pr-check pipeline

2

Turn on five policies on main – build validation = pr-check

3

Prove it: a direct push to main fails with TF402455

4

Bug: branch, push a failing test, open the PR – check goes red

5

Fix it; buddy approves; push once more and watch the approval reset

6

Re-approve, squash-complete; bug set to Resolved

Use your own work item IDs. Review your buddy's PR in their organisation. About 30 minutes.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

CASE STUDY

What Done Looks Like

Policies	Pull request	History
Five policies on main; a direct push is refused with TF402455.	Red check on the failing test, green after the fix; approval reset by the extra push, then re-approved; thread resolved.	One squash commit on main with ‘Related work items: #<bug-id>’; bug Resolved.

Done when: main changes only through a reviewed, linked, validated pull request – and you’ve seen each rule stop you once.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

SYMPTOM → CAUSE → FIX

Lab Issues You Are Most Likely to Hit

Symptom	Cause	Fix
Buddy not found when adding a user	Partial name or typo	Use the full UPN …@labasservice.onmicrosoft.com
'Complete' is disabled	A required policy isn't met	Read the policy list at the top of the PR
PR check didn't start	PR still a draft, Trigger Manual, or a path filter	Publish the PR; edit the policy
Approval disappeared	Votes reset on the new push (by design)	Ask your buddy to re-approve
Can't approve your own PR	Self-approval switched off (by design)	Your buddy approves
No pipeline to pick in the policy	pr-check not created yet	Create it from /pipelines/pr-check.yml

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

8

Knowledge Check

Seven scenario questions – the details decide the answer

© NIIT, All Information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · QUESTION 1 OF 7

Question 1

main requires 1 reviewer with “When new changes are pushed: Reset all approval votes”. Asha approves at 10:00. At 10:05 the author pushes a one-word typo fix; build validation passes at 10:12. What happens when the author clicks Complete at 10:15?

A. It’s blocked – Asha’s approval was reset by the 10:05 push and a new approval is needed

B. It completes – the build passed on the latest code, which satisfies the policy

C. It completes – typo-only pushes don’t reset votes

D. It’s blocked until the build expires and runs again

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 1

Answer 1: A

A **Correct – It's blocked – Asha's approval was reset by the 10:05 push and a new approval is needed**
Any push after approval resets approval votes under this option – the reviewer must approve the new iteration.

B **Not this – It completes – the build passed on the latest code, which satisfies the policy**
Build validation and reviewer approval are separate policies; a green build doesn't restore the vote.

C **Not this – It completes – typo-only pushes don't reset votes**
Azure Repos doesn't inspect the size of a push; every new push resets approvals.

D **Not this – It's blocked until the build expires and runs again**
The build is fresh and green; expiration isn't the blocker here.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

Question 2

Your PR from bugfix/57 into main has a green build validation. Since you branched, main has received three other merges. What code did the PR build actually compile and test?

A. The tip of bugfix/57 only

B. main only, to check that it still builds

C. A temporary merge of bugfix/57 into main as it was when the build ran

D. bugfix/57 rebased onto main by the build agent

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 2

Answer 2: C

A **Not this – The tip of bugfix/57 only**
The build doesn't test your branch in isolation.

B **Not this – main only, to check that it still builds**
Building main alone would tell you nothing about the PR.

C **Correct – A temporary merge of bugfix/57 into main as it was when the build ran**
Azure Repos builds refs/pull/<id>/merge – your branch merged with the current target.

D **Not this – bugfix/57 rebased onto main by the build agent**
No rebase happens; a merge commit is created for the build.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · QUESTION 3 OF 7

Question 3

A team removes the Build validation branch policy and adds this to azure-pipelines.yml instead: pr: branches: include: [main] — the code is in Azure Repos. What happens when the next pull request into main is opened?

A. The pipeline runs for the PR, exactly as the policy used to

B. No PR build runs – Azure Repos ignores the YAML pr trigger

C. The pipeline runs but its result is optional

D. The pipeline runs only after the PR is completed

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 3

Answer 3: B

A **Not this – The pipeline runs for the PR, exactly as the policy used to**
That's how it works for GitHub and Bitbucket – not Azure Repos.

B **Correct – No PR build runs – Azure Repos ignores the YAML pr trigger**
For Azure Repos, PR builds come only from the Build validation policy.

C **Not this – The pipeline runs but its result is optional**
Nothing runs at PR time, required or optional.

D **Not this – The pipeline runs only after the PR is completed**
A run after completion comes from the CI trigger on main – not from the pr section.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · QUESTION 4 OF 7

Question 4

A team wants every commit on main to be a complete, reviewed change that links to its work item, and wants to undo a whole PR with one plain “git revert <sha>”. Which merge type fits best?

A Basic merge (no fast-forward)

B Rebase and fast-forward

C Rebase with merge commit (semi-linear)

D Squash merge

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 4

Answer 4: D

A **Not this – Basic merge (no fast-forward)**
Branch commits (including WIP) land on main, and reverting the merge needs 'git revert -m 1'.

B **Not this – Rebase and fast-forward**
Every individual branch commit lands on main – no single commit to revert.

C **Not this – Rebase with merge commit (semi-linear)**
Keeps every branch commit on main; reverting needs '-m 1' on the merge commit.

D **Correct – Squash merge**
One commit per PR, carrying 'Merged PR n' and 'Related work items' – revert it with one plain command.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

KNOWLEDGE CHECK · QUESTION 5 OF 7

Question 5

Production runs release/2.3, cut three weeks ago. main has moved on. A bug is found that also exists in main. Following 'upstream first' (Microsoft's Release Flow), where is the fix made first?

A On release/2.3, then release/2.3 is merged back into main

B On a hotfix branch from release/2.3, merged only into release/2.3

C In main through a PR, then cherry-picked into release/2.3 through a PR

D Directly on release/2.3, then cherry-picked into main

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 5

Answer 5: C

A **Not this – On release/2.3, then release/2.3 is merged back into main**
Merging a release branch back drags release-only changes into main.

B **Not this – On a hotfix branch from release/2.3, merged only into release/2.3**
Fixing only the release branch lets the next release from main bring the bug back.

C **Correct – In main through a PR, then cherry-picked into release/2.3 through a PR**
Fix main first, then cherry-pick – the fix can never be lost from future releases.

D **Not this – Directly on release/2.3, then cherry-picked into main**
That's 'downstream first' – if the cherry-pick is forgotten, the bug returns.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · QUESTION 6 OF 7

Question 6

Policy: minimum 1 reviewer, "Allow requestors to approve their own changes" OFF, nothing else ticked. Ravi opens a PR. Meena pushes the final commit to Ravi's branch to fix a test, then Meena approves. Can it complete – and which option would stop this?

A. Yes – add "Prohibit the most recent pusher from approving their own changes" to stop it

B. No – Meena became a requestor when she pushed

C. Yes – only "Reset all code reviewer votes" would stop it

D. No – the comment-resolution policy blocks it

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 6

Answer 6: A

A **Correct – Yes – add "Prohibit the most recent pusher from approving their own changes" to stop it**
Meena approved code she wrote herself; this option exists precisely to block that.

B **Not this – No – Meena became a requestor when she pushed**
Pushing doesn't make someone the PR's requestor; the creator is Ravi.

C **Not this – Yes – only "Reset all code reviewer votes" would stop it**
Resetting votes happens on pushes; Meena voted after her push, so her vote stands.

D **Not this – No – the comment-resolution policy blocks it**
No comment threads are involved in the scenario.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

NIIT ENTERPRISE

KNOWLEDGE CHECK · QUESTION 7 OF 7

Question 7

Every PR into main needs at least one approval. For changes under /infra/, at least one approval must come from the Platform team – a single Platform approval is enough. What is the simplest configuration on main?

A. Minimum reviewers = 2 for every pull request

B. Move /infra into a separate repository with its own policies

C. Deny ‘Contribute’ on /infra for everyone except the Platform team

D. Minimum reviewers = 1, plus an automatically included reviewer: Platform team, Required, path /infra/*

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

KNOWLEDGE CHECK · ANSWER 7

Answer 7: D

A **Not this – Minimum reviewers = 2 for every pull request**
Two reviewers everywhere slows every PR and still doesn't guarantee Platform reviews infra.

B **Not this – Move /infra into a separate repository with its own policies**
Possible, but a big restructuring for a review rule.

C **Not this – Deny 'Contribute' on /infra for everyone except the Platform team**
Permissions on folders aren't how Git repos in Azure Repos work – and it blocks contributions instead of requiring review.

D **Correct – Minimum reviewers = 1, plus an automatically included reviewer: Platform team, Required, path /infra/***
Path-filtered required reviewers add Platform only when /infra/* changes.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

WRAP-UP

Recap

Topic	Key takeaway
Git fundamentals	Commits are snapshots, branches are pointers; only pushed commits exist for the team
Branching models	Short branches into a protected main by default; release branches fix upstream first
Pull requests	Small PRs with a description and work item; review intent → correctness → tests first
Branch policies	No self-approval, votes reset on push, work items, resolved comments, build on the merge
Merge strategies	Squash: one reviewed, linked, revertable commit per PR – enforced by policy
History hygiene	Never rewrite shared history; force-with-lease only on your own branch; tag releases
What's new	Copilot code review for Azure Repos (preview), PR status on cards, MCP Server – humans decide

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.